



Ben-Gurion University of the Negev

Faculty of Engineering

Department of Mechanical Engineering



Introduction to Image Processing Course – Second Assignment

Student name: Tamir Levin, 315765347

Date: 12.1.2025

Assistant Lecturer's Name: Iaroslav Grozdov

1. Environment Setup and Preparations

1.1. Library versions

```
print('pandas version: ' + pd.__version__)  
print('numpy version: ' + np.__version__)  
print('tensorflow version: ' + tf.__version__)  
print('scikit-image version: ' + ski.__version__)
```

[2] ✓ 0.0s

Python

```
... pandas version: 2.0.3  
... numpy version: 1.26.0  
... tensorflow version: 2.10.0  
... scikit-image version: 0.19.3
```

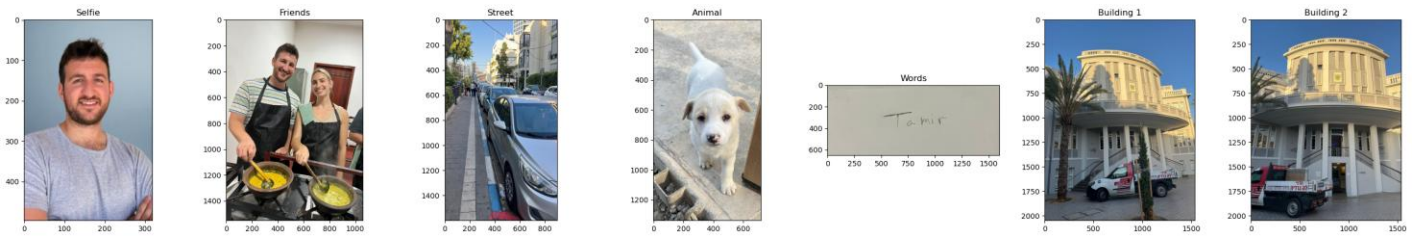
```
conda_env = os.environ.get('CONDA_DEFAULT_ENV')  
print(f"current Conda environment: '{conda_env}'.")
```

[36]

Python

```
... current Conda environment: 'image_processing_env'.
```

1.2. Images preparation for the course



2. Low-pass and High-pass filters

Image filters and edge detection are essential image processing techniques that rely on several key parameters to effectively identify features and boundaries in digital images by applying mathematical operations to pixel values, allowing us to enhance features, reduce noise, or detect specific patterns in visual data.

1.3. Image Filters

1.3.1. Kernel size

The kernel size determines how many surrounding pixels influence each calculation. A larger kernel creates stronger smoothing effects but requires more computational power, while a smaller kernel preserves more detail but may be less effective at noise reduction.

1.3.2. Kernel shape

The arrangement and weights of values in the kernel matrix define the filter's behavior. Different patterns can emphasize horizontal edges, blur uniformly, or sharpen specific features according to the chosen configuration.

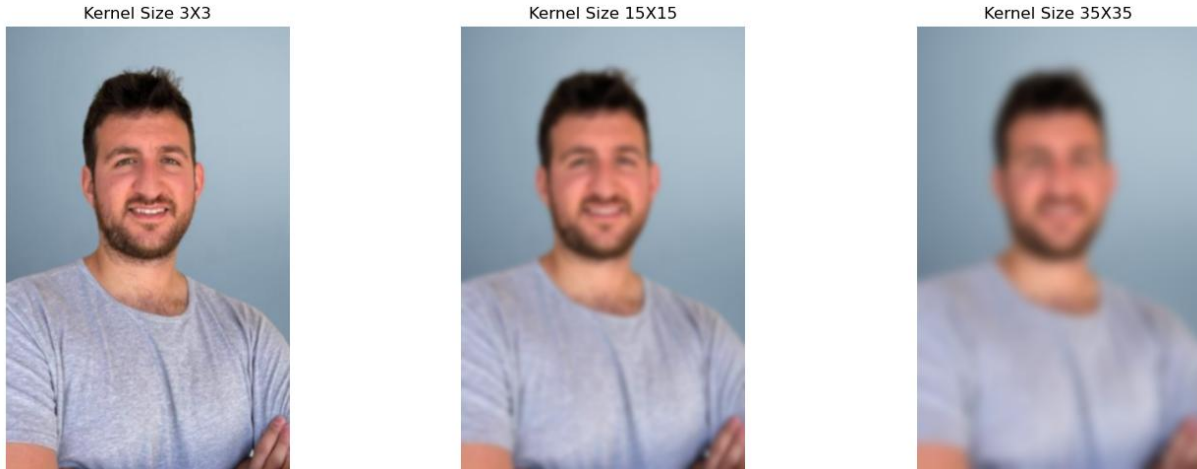
1.4. Edge Detection

1.4.1. Thresholds

Threshold settings determine how sensitive the detector is to intensity changes in the image. Lower thresholds detect more subtle edges but may include noise, while higher thresholds identify only the strongest boundaries.

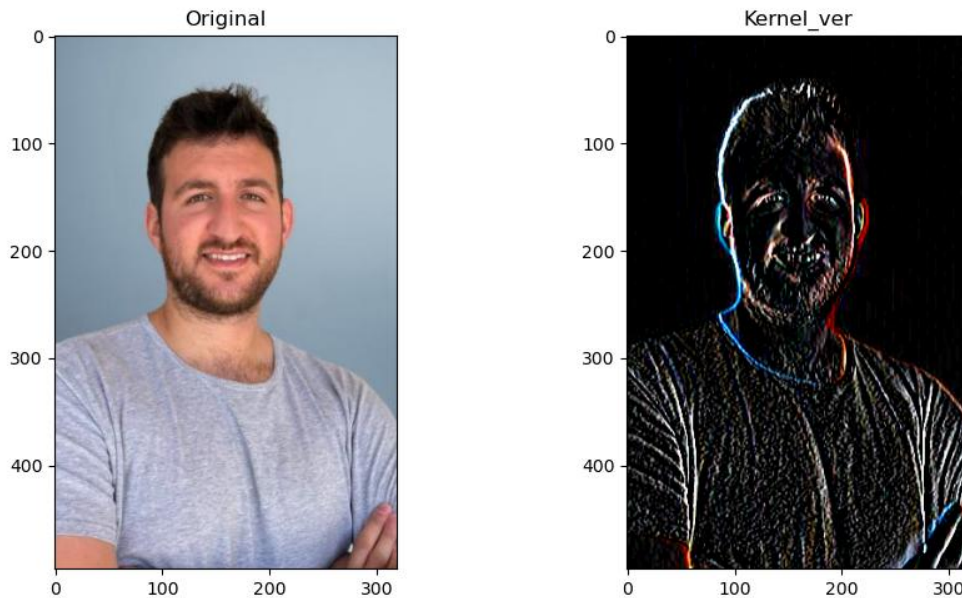
1.5. Blurring images using different sizes of Gaussian kernels

In this section, the selfie image was blurred using Low-Pass Gaussian kernels in 3 different sizes. The result is shown below.



1.6. Blurring images using a given kernel_ver

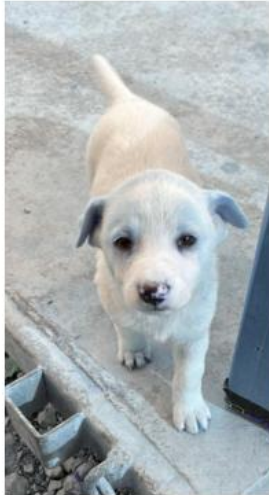
In this section, the same image was blurred using a given kernel: $\begin{bmatrix} 0 & 1 & -1 & 0 \\ 1 & 3 & -3 & -1 \\ 1 & 3 & -3 & 1 \\ 0 & 1 & -1 & 0 \end{bmatrix}$. The result is shown below.



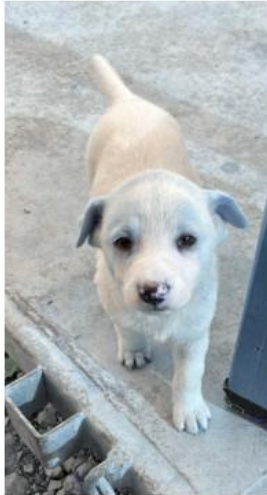
1.7. Median, Mean and Low-Pass Filters

In this section, the animal image is processed using median and mean low-pass filters. The results shown below demonstrate how these different filtering approaches and different kernels affect the output.

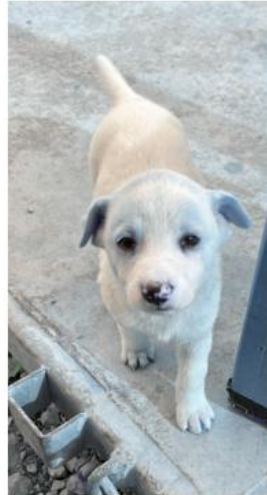
Gaussian Blur (Kernel=3)



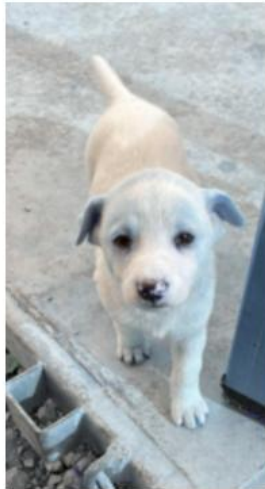
Mean Blur (Kernel=3)



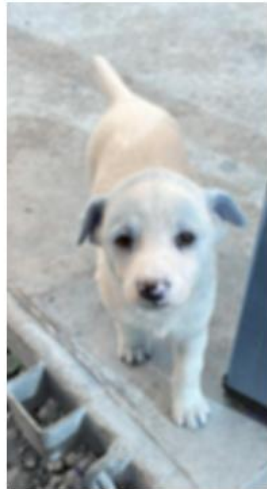
Median Blur (Kernel=3)



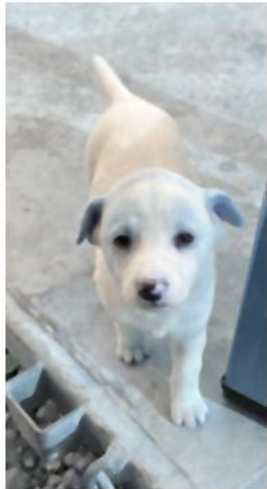
Gaussian Blur (Kernel=15)



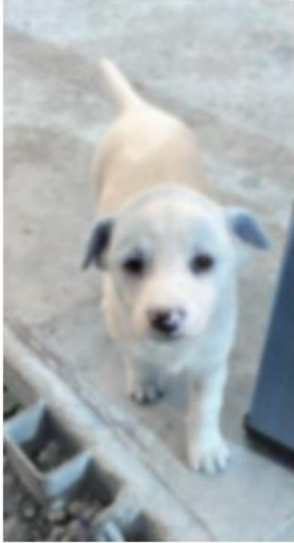
Mean Blur (Kernel=15)



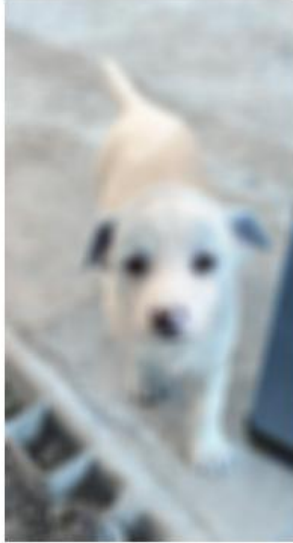
Median Blur (Kernel=15)



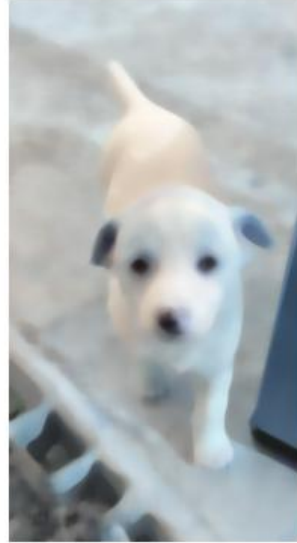
Gaussian Blur (Kernel=35)



Mean Blur (Kernel=35)



Median Blur (Kernel=35)



Each filter affects the image differently. As the kernel size gets bigger, the blurring becomes stronger. Gaussian blurring creates a smoother and more refined effect compared to the others. Mean blurring makes the image look a bit more smeared in certain areas. Median blurring, on the other hand, creates strong blurring within regions but keeps the edges sharp.

1.8. Canny edge-detection algorithm

In this section, the Canny edge-detection algorithm was applied to the image using three different sets of threshold values. The result is shown below.

Canny (50, 150)



Canny (100, 200)



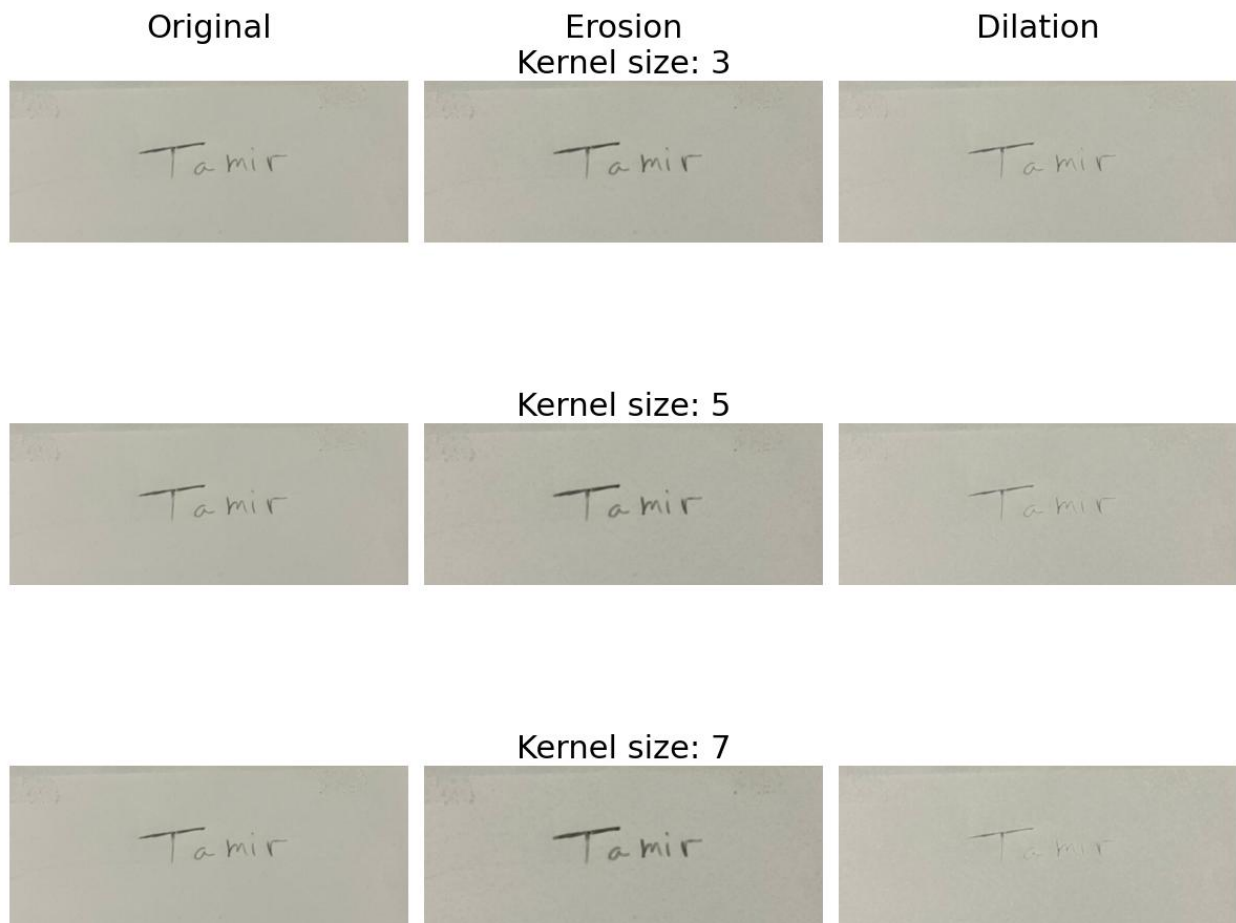
Canny (150, 250)



3. Morphological operations

1.9. Erosion and Dilation

Erosion and dilation were performed on the image to observe their effects. Erosion reduces the boundaries of objects, while dilation expands them. The processed images are displayed below for comparison.



1.10. Opening, Closing and Morphological gradient

Opening, closing, and morphological gradient operations were applied to further analyze the effects of combining erosion and dilation in different ways. Opening helps remove small noise, closing fills gaps, and the morphological gradient highlights the edges of objects. The results are presented below.



4. Harris Corner Detector and Feature Matching

1.11. Manually applied Harris Corner Detector function

The manually written Harris Corner Detector function is shown below.

```
# Defined parameters
window_size = 3
k = 0.04
threshold_factor = 0.1

# Convert to grayscale
building_1_gray = cv2.cvtColor(building_1, cv2.COLOR_BGR2GRAY)
gray_float32 = np.float32(building_1_gray)

# Gaussian Blur
building_1_gaussian = cv2.GaussianBlur(gray_float32, (3, 3), 0)

# Sobel derivatives
dx = cv2.Sobel(building_1_gaussian, cv2.CV_64F, 1, 0, ksize=3)
dy = cv2.Sobel(building_1_gaussian, cv2.CV_64F, 0, 1, ksize=3)
dx2 = dx**2
dy2 = dy**2
dxy = dx * dy

# Sum of products within a window (using convolution for efficiency)
kernel = np.ones((window_size, window_size))

Sx2 = cv2.filter2D(dx2, -1, kernel)
Sy2 = cv2.filter2D(dy2, -1, kernel)
Sxy = cv2.filter2D(dxy, -1, kernel)

# Harris response calculation
det_H = (Sx2 * Sy2) - (Sxy**2)
trace_H = Sx2 + Sy2
matrix_R = det_H - k * (trace_H**2)

# Threshold and mark corners
threshold = threshold_factor * matrix_R.max()
building_1_corners = building_1.copy()

# Use vectorized operations to find corners
corners = np.argwhere(matrix_R > threshold)
for y, x in corners:
    cv2.circle(building_1_corners, (x, y), 2, (255, 0, 0), -1)
```

1.12. Manually applied function compared to the built-in OpenCV Harris Corner Detector function

Corner Detector function

In this section, the Harris Corner Detector was applied to the image in two ways: first by the manually written function, and then using the built-in `cv2.cornerHarris` function. The results from both methods are shown below, along with a discussion on any noticeable differences in accuracy, resolution, or the appearance of detected corners.

Manually Implemented Harris



Harris by CV2 Function



To ensure proper functionality, the input image was converted to grayscale and its type set to `float32`. The results from both methods are compared to highlight any differences in the outputs.

- **Manual Implementation:** The Harris Corner Detector was manually implemented by calculating the gradients, structure tensor, and response function step-by-step. This approach provides a deeper understanding of how the algorithm works internally.
- **Using `cv2.cornerHarris`:** The same detection was performed using the OpenCV function, which simplifies the process by automatically handling all the steps.

As seen, different corners of the same images were detected for each function.

1.13. Shi-Tomasi Detector

In this section, the Shi-Tomasi Corner Detector was applied to the same image to identify corners based on the "good features to track" OpenCV built-in function. Various corner selection parameters were tested, including the number of corners to detect, the quality level, and the minimum distance between detected corners.

Distance: 10, Max Corners: 50



Distance: 10, Max Corners: 400



Distance: 10, Max Corners: 1000



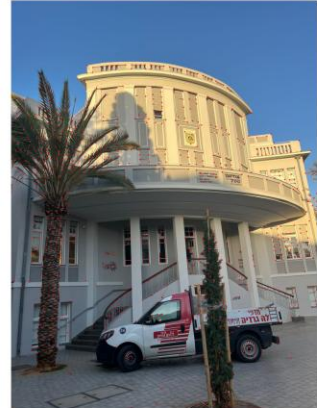
Distance: 20, Max Corners: 50



Distance: 20, Max Corners: 400



Distance: 20, Max Corners: 1000



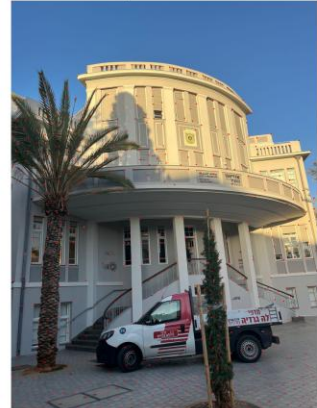
Distance: 40, Max Corners: 50



Distance: 40, Max Corners: 400



Distance: 40, Max Corners: 1000



The results demonstrate how changing these parameters influences the detection outcome:

- **Number of Corners:** Adjusting this parameter controls how many corners are detected. A higher number detects more features, while a lower number focuses on the most prominent ones.
- **Quality Level:** This parameter determines the minimum accepted quality of corners, with higher values selecting only the strongest corners and lower values allowing more corners of varying quality.
- **Minimum Distance:** This ensures that corners are not clustered too closely, influencing the spacing between detected features

1.14. FAST and ORB feature matching

In this section, the FAST algorithm was applied to the building images. The result of each image key points is shown below.

1.14.1. FAST Keypoints

Keypoints on Image 1

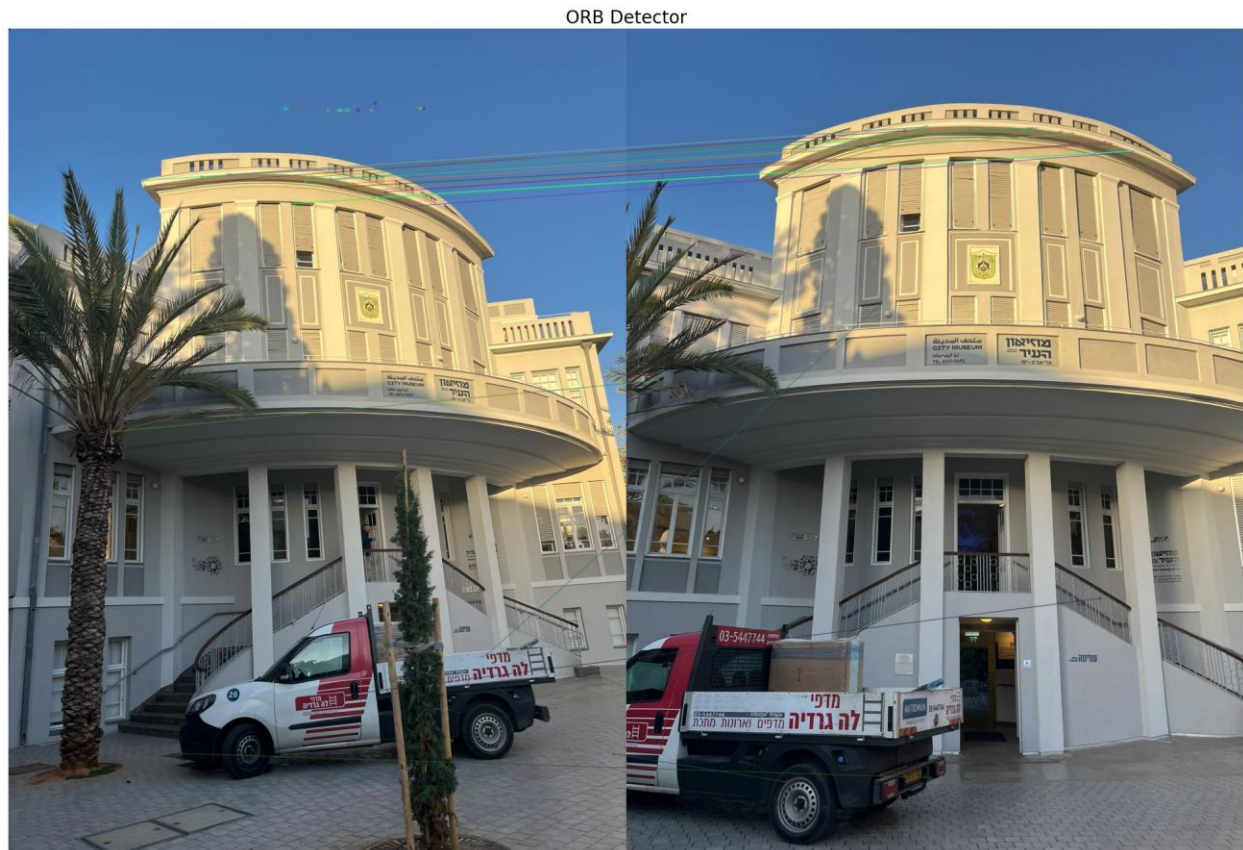


Keypoints on Image 2



1.14.2. ORB Feature Matching

This section explores feature detection and matching using FAST and ORB algorithms. Two images with similar content captured from different viewpoints were used for this task. The result is shown below.



1.14.3. BRISK Feature Matching

For the third implementation I chose the BRISK algorithm. BRISK combines a binary descriptor with scale and rotation-invariant key point detection. It is particularly effective for detecting and matching features in images with scale variations. The result is shown below.

